

CS251A - Advanced Computer Architecture

HW2 - Magic Instructions

Summary

The idea was to implement a performance profiler in gem5 using the magic instruction (`fsubr`), that acts as a hardware equivalent of a debug statement. When executed, it prints key performance metrics: IPC, idle rate, stall rate, MPKI, and branch prediction accuracy, by accessing private statistics from each pipeline stage (fetch, decode, rename, execute, and commit). This will allow developers to insert the instruction at different points in the code to analyze the performance impact of specific functions or code regions in real-time.

Results

The following program tests the basic functionality of the instruction - is it properly processed by each pipeline stage or not?

```
#include <stdio.h>
int main() {
    float a = 3.0, b = 1.0;
    float result = b-a;
    asm volatile ("fsubr %%st, %%st" ::: "st");
    return 0;
}
```

gem5 debug output confirms the successful execution:

```
68138127: board.processor.cores.core.fetch: Magic instruction fetched at cycle 68138127 [tid:0, sn:272809]
68138460: board.processor.cores.core.decode: Magic instruction decoded at cycle 68138460 [tid:0, sn:272809]
68138793: board.processor.cores.core.rename: Magic instruction renamed at cycle 68138793 [tid:0, sn:272809]
68139459: board.processor.cores.core.iew: Magic instruction dispatched at cycle 68139459 [tid:0, sn:272809]
68215050: board.processor.cores.core.fetch: Magic instruction fetched at cycle 68215050 [tid:0, sn:272858]
68216715: board.processor.cores.core.decode: Magic instruction decoded at cycle 68216715 [tid:0, sn:272858]
68217048: board.processor.cores.core.rename: Magic instruction renamed at cycle 68217048 [tid:0, sn:272858]
68217714: board.processor.cores.core.iew: Magic instruction dispatched at cycle 68217714 [tid:0, sn:272858]
68219712: board.processor.cores.core.commit: Magic instruction committed at cycle 68219712
```

The `MAGIC` instruction is encountered multiple times in the pipeline due to speculative execution and misprediction flushes. But it is finally committed only once, confirming its successful processing by each pipeline stage.

The following program gives a practical use case for this instruction. There are multiple loops, each exhibiting a different workload characteristic. Seeing the performance start after each will help see the efficiency of the CPU against different challenges.

```

#include <stdio.h>
int main() {
    int sum = 0;
    for (int i = 0; i < 100000; i++) {
        sum += i;
    }
    asm volatile ("fsubr %%st, %%st" ::: "st");
    float result = 1.0;
    for (int i = 1; i <= 50000; i++) {
        result = (result + i) * 1.00001f;
    }
    asm volatile ("fsubr %%st, %%st" ::: "st");
    int x = 0;
    for (int i = 0; i < 100000; i++) {
        if (i % 3 == 0) x += 3;
        else if (i % 3 == 1) x -= 2;
        else x += 1;
    }
    asm volatile ("fsubr %%st, %%st" ::: "st");
}

```

gem5 output shows the changes in the statistics after each loop, which can be attributed to different microarchitectural parameters and can help further optimize the design:

```

334561770: board.processor.cores.core:
Magic Profiler:
> IPC=0.582844
> Idle Rate=0.0609262
> Stall Rate=0.0406483
> MPKI=2.70844
> Predictor Accuracy=0.985662

600996069: board.processor.cores.core:
Magic Profiler:
> IPC=0.601501
> Idle Rate=0.0339163
> Stall Rate=0.0226707
> MPKI=1.46373
> Predictor Accuracy=0.990111

913373379: board.processor.cores.core:
Magic Profiler:
> IPC=1.41661
> Idle Rate=0.0223168
> Stall Rate=0.0149869
> MPKI=0.412551
> Predictor Accuracy=0.99625

```